

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 1 – Industrial Problem-Solving**

Course Code:	CSA0901	Course Title:	Programming in Java
CO Assessed:	CO4 – Precision (BL3,BL4)	Assessment:	ASSESSMENT TOOL 1 - Industrial Problem-Solving
Weightage:	40%	Total Marks:	100
CO Statement:	4 Design GUI-based user interfaces using Abstract Window Toolkit, Swing, and Applets by implementing event handling, layout managers, AWT controls, and menus.		
Student Name:		Reg. No.:	
Section Batch:	/ SLOT B	Date:	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 4 questions Total: 100 Marks.
- No negative marking. Un answered questions carry zero marks.
- No calculators or electronic devices permitted.
- Duration: 30 minutes.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Application of Concepts	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Analytical & Decision-Making Skills	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Solution Design & Approach	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Clarity, Justification & Presentation	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Industrial Production Control Applet

Develop a Java applet for a manufacturing plant that monitors **multiple machines simultaneously**. Provide controls to Start, Stop, Pause, and Reset each machine. Use the **Delegation Event Model** with appropriate event classes and listener interfaces. Design the interface using suitable **AWT layout managers**, and provide a **MenuBar** for machine selection, monitoring, and report generation. The applet must dynamically update machine status and prevent invalid operations such as starting an already running machine.

2. Banking Transaction Management Applet

Develop an interactive banking applet supporting **Deposit, Withdrawal, Fund Transfer, and Balance Inquiry**. Use AWT controls and menus to accept transaction details. Implement event handling with appropriate listener interfaces and validate conditions such as insufficient balance, invalid amount, and incomplete input. Use different layout managers to organize the transaction panels and display transaction status dynamically.

3. Hospital Emergency Management Applet

Design a hospital emergency-management applet that accepts **patient registration, emergency category, doctor selection, and treatment priority**. Use Choice, Checkbox, TextField, TextArea, Button, and menus. Implement event handling to automatically assign priority and display alerts for critical patients. The system must prevent duplicate patient registration and validate all required inputs before confirmation.

4. Smart Factory Monitoring and Alert Applet

Develop a Java applet for monitoring a smart factory containing **temperature, pressure, machine status, and production-level controls**. Provide Normal, Warning, and Emergency modes using AWT components. Implement multiple event listeners to respond to user actions and generate appropriate alerts. Use **BorderLayout, GridLayout, CardLayout, and menus** to create different monitoring sections. The applet should dynamically switch between views and prevent conflicting machine operations.

1. Industrial Production Control Applet

Aim

To develop a Java Applet for monitoring and controlling multiple machines using the **Delegation Event Model**, AWT controls, layout managers, and `MenuBar`.

Program

```
import java.applet.Applet; import java.awt.*; import java.awt.event.*;
public class IndustrialProductionApplet extends Applet
    implements ActionListener {
```

```

Button start1, stop1, pause1, reset1;
Button start2, stop2, pause2, reset2;
Button start3, stop3, pause3, reset3;

Label status1, status2, status3;
TextArea report;

MenuBar menuBar;
Menu machineMenu, monitorMenu, reportMenu;

public void init() {

    setLayout(new BorderLayout());

    // Machine 1
    Panel machine1 = new Panel(new GridLayout(2, 5));

    machine1.add(new Label("Machine 1"));

    start1 = new Button("Start");
    stop1 = new Button("Stop");
    pause1 = new Button("Pause");
    reset1 = new Button("Reset");

    status1 = new Label("Status: Stopped");

    machine1.add(start1);
    machine1.add(stop1);
    machine1.add(pause1);
    machine1.add(reset1);
    machine1.add(status1);

    // Machine 2
    Panel machine2 = new Panel(new GridLayout(2, 5));

    machine2.add(new Label("Machine 2"));

    start2 = new Button("Start");
    stop2 = new Button("Stop");
    pause2 = new Button("Pause");
    reset2 = new Button("Reset");

    status2 = new Label("Status: Stopped");

    machine2.add(start2);
    machine2.add(stop2);
    machine2.add(pause2);
    machine2.add(reset2);
    machine2.add(status2);

    // Machine 3
    Panel machine3 = new Panel(new GridLayout(2, 5));

    machine3.add(new Label("Machine 3"));

    start3 = new Button("Start");
    stop3 = new Button("Stop");
    pause3 = new Button("Pause");
    reset3 = new Button("Reset");

    status3 = new Label("Status: Stopped");

    machine3.add(start3);

```

```

machine3.add(stop3);
machine3.add(pause3);
machine3.add(reset3);
machine3.add(status3);

Panel machines = new Panel(new GridLayout(3, 1));

machines.add(machine1);
machines.add(machine2);
machines.add(machine3);

add(machines, BorderLayout.CENTER);

// Report area
report = new TextArea(5, 40);
report.setEditable(false);

add(report, BorderLayout.SOUTH);

// Event listeners
start1.addActionListener(this);
stop1.addActionListener(this);
pause1.addActionListener(this);
reset1.addActionListener(this);

start2.addActionListener(this);
stop2.addActionListener(this);
pause2.addActionListener(this);
reset2.addActionListener(this);

start3.addActionListener(this);
stop3.addActionListener(this);
pause3.addActionListener(this);
reset3.addActionListener(this);

// MenuBar
menuBar = new MenuBar();

machineMenu = new Menu("Machine");
monitorMenu = new Menu("Monitoring");
reportMenu = new Menu("Report");

MenuItem machine1Item =
    new MenuItem("Machine 1");
MenuItem machine2Item =
    new MenuItem("Machine 2");
MenuItem machine3Item =
    new MenuItem("Machine 3");

MenuItem monitorItem =
    new MenuItem("View Status");

MenuItem reportItem =
    new MenuItem("Generate Report");

machineMenu.add(machine1Item);
machineMenu.add(machine2Item);
machineMenu.add(machine3Item);

monitorMenu.add(monitorItem);
reportMenu.add(reportItem);

```

```

        menuBar.add(machineMenu);
        menuBar.add(monиторMenu);
        menuBar.add(reportMenu);

        setMenuBar(menuBar);

        machine1Item.addActionListener(this);
        machine2Item.addActionListener(this);
        machine3Item.addActionListener(this);
        мониторItem.addActionListener(this);
        reportItem.addActionListener(this);
    }

    public void actionPerformed(ActionEvent e) {

        Object source = e.getSource();

        // Machine 1
        if (source == start1) {
            changeStatus(status1, "Running");
        }
        else if (source == stop1) {
            changeStatus(status1, "Stopped");
        }
        else if (source == pause1) {
            changeStatus(status1, "Paused");
        }
        else if (source == reset1) {
            changeStatus(status1, "Stopped");
        }

        // Machine 2
        else if (source == start2) {
            changeStatus(status2, "Running");
        }
        else if (source == stop2) {
            changeStatus(status2, "Stopped");
        }
        else if (source == pause2) {
            changeStatus(status2, "Paused");
        }
        else if (source == reset2) {
            changeStatus(status2, "Stopped");
        }

        // Machine 3
        else if (source == start3) {
            changeStatus(status3, "Running");
        }
        else if (source == stop3) {
            changeStatus(status3, "Stopped");
        }
        else if (source == pause3) {
            changeStatus(status3, "Paused");
        }
        else if (source == reset3) {
            changeStatus(status3, "Stopped");
        }

        // Menu operations
        else if (e.getActionCommand().equals("View Status")) {
            showStatus();
        }
    }

```

```

        else if (e.getActionCommand().equals("Generate Report")) {
            generateReport();
        }
        else {
            report.append(e.getActionCommand() +
                " selected\n");
        }
    }

    void changeStatus(Label label, String newStatus) {

        String oldStatus = label.getText();

        if (newStatus.equals("Running") &&
            oldStatus.equals("Status: Running")) {

            report.append(
                "Invalid operation: Machine already running\n");
            return;
        }

        label.setText("Status: " + newStatus);

        report.append(
            "Machine status changed to " +
            newStatus + "\n");
    }

    void showStatus() {

        report.append("\n--- MACHINE STATUS ---\n");
        report.append(status1.getText() + "\n");
        report.append(status2.getText() + "\n");
        report.append(status3.getText() + "\n");
    }

    void generateReport() {

        report.append("\n--- PRODUCTION REPORT ---\n");
        report.append(status1.getText() + "\n");
        report.append(status2.getText() + "\n");
        report.append(status3.getText() + "\n");
        report.append("-----\n");
    }
}

```

Input

No keyboard input is required.

User interacts with:

```

Start
Stop
Pause
Reset
Machine Menu
Monitoring Menu
Report Menu

```

Sample Output

```

Machine status changed to Running
Machine status changed to Paused

```

Machine status changed to Stopped

--- MACHINE STATUS ---

Status: Stopped

Status: Running

Status: Paused

--- PRODUCTION REPORT ---

Status: Stopped

Status: Running

Status: Paused

Key Concepts

Applet

Button

Label

TextArea

MenuBar

Menu

MenuItem

ActionListener

ActionEvent

Delegation Event Model

BorderLayout

GridLayout

Multiple machine control

Input/operation validation

2. Banking Transaction Management Applet

Aim

To develop an interactive Java Applet for performing **Deposit, Withdrawal, Fund Transfer, and Balance Inquiry** using AWT controls and event handling.

Program

```
import java.applet.Applet;import java.awt.*;import java.awt.event.*;
public class BankingApplet extends Applet
    implements ActionListener {

    TextField amountField;
    TextField transferField;

    Button depositButton;
    Button withdrawButton;
    Button transferButton;
    Button balanceButton;

    Label balanceLabel;
    TextArea status;

    double balance = 5000;

    public void init() {
```



```

setLayout(new BorderLayout());

Panel inputPanel = new Panel(new GridLayout(4, 2));

inputPanel.add(new Label("Amount:"));
amountField = new TextField();
inputPanel.add(amountField);

inputPanel.add(new Label("Transfer Amount:"));
transferField = new TextField();
inputPanel.add(transferField);

depositButton = new Button("Deposit");
withdrawButton = new Button("Withdraw");
transferButton = new Button("Transfer");
balanceButton = new Button("Balance Inquiry");

inputPanel.add(depositButton);
inputPanel.add(withdrawButton);
inputPanel.add(transferButton);
inputPanel.add(balanceButton);

add(inputPanel, BorderLayout.NORTH);

balanceLabel =
    new Label("Balance: Rs. " + balance);

add(balanceLabel, BorderLayout.CENTER);

status = new TextArea(8, 40);
status.setEditable(false);

add(status, BorderLayout.SOUTH);

depositButton.addActionListener(this);
withdrawButton.addActionListener(this);
transferButton.addActionListener(this);
balanceButton.addActionListener(this);
}

public void actionPerformed(ActionEvent e) {
    try {
        if (e.getSource() == depositButton) {
            String text = amountField.getText();

            if (text.isEmpty()) {
                status.setText("Enter deposit amount");
                return;
            }

            double amount =
                Double.parseDouble(text);

            if (amount <= 0) {
                status.setText(
                    "Invalid deposit amount");
                return;
            }
        }
    }
}

```

```

        balance += amount;

        updateBalance();

        status.setText(
            "Deposit successful: Rs. " + amount);
    }

    else if (e.getSource() == withdrawButton) {

        String text = amountField.getText();

        if (text.isEmpty()) {
            status.setText("Enter withdrawal amount");
            return;
        }

        double amount =
            Double.parseDouble(text);

        if (amount <= 0) {
            status.setText(
                "Invalid withdrawal amount");
            return;
        }

        if (amount > balance) {
            status.setText(
                "Insufficient balance");
            return;
        }

        balance -= amount;

        updateBalance();

        status.setText(
            "Withdrawal successful: Rs. " +
            amount);
    }

    else if (e.getSource() == transferButton) {

        String text = transferField.getText();

        if (text.isEmpty()) {
            status.setText(
                "Enter transfer amount");
            return;
        }

        double amount =
            Double.parseDouble(text);

        if (amount <= 0) {
            status.setText(
                "Invalid transfer amount");
            return;
        }

        if (amount > balance) {
            status.setText(
                "Insufficient balance");
            return;
        }
    }

```

```

    }

    balance -= amount;

    updateBalance();

    status.setText(
        "Fund transfer successful: Rs. " +
        amount);
}

else if (e.getSource() == balanceButton) {

    status.setText(
        "Current Balance: Rs. " + balance);
}

}

catch (NumberFormatException ex) {

    status.setText(
        "Invalid amount. Enter numbers only.");
}

}

void updateBalance() {

    balanceLabel.setText(
        "Balance: Rs. " + balance);
}

}

```

Input

Amount: 2000

Output

Deposit successful: Rs. 2000.0
Balance: Rs. 7000.0

For insufficient balance:

Insufficient balance

3. Hospital Emergency Management Applet

Aim

To develop a Java Applet for patient registration, emergency category, doctor selection, and automatic treatment priority using AWT controls and event handling.

Program

```

import java.applet.Applet;import java.awt.*;import java.awt.event.*;
public class HospitalApplet extends Applet
    implements ActionListener {

    TextField patientName;
    Choice category;
    Choice doctor;

```

```

Checkbox critical;
Button registerButton;

TextArea output;

String registeredPatient = "";

public void init() {

    setLayout(new BorderLayout());

    Panel form = new Panel(new GridLayout(5, 2));

    form.add(new Label("Patient Name:"));

    patientName = new TextField();
    form.add(patientName);

    form.add(new Label("Emergency Category:"));

    category = new Choice();
    category.add("General");
    category.add("Serious");
    category.add("Critical");

    form.add(category);

    form.add(new Label("Doctor:"));

    doctor = new Choice();
    doctor.add("Dr. Kumar");
    doctor.add("Dr. Priya");
    doctor.add("Dr. Arun");

    form.add(doctor);

    form.add(new Label("Critical Patient:"));

    critical = new Checkbox("Yes");
    form.add(critical);

    registerButton =
        new Button("Register Patient");

    form.add(registerButton);

    add(form, BorderLayout.NORTH);

    output = new TextArea(10, 40);
    output.setEditable(false);

    add(output, BorderLayout.CENTER);

    registerButton.addActionListener(this);
}

public void actionPerformed(ActionEvent e) {

    String name = patientName.getText().trim();

    if (name.isEmpty()) {

        output.setText(

```

```

        "Error: Patient name is required.");
    return;
}

if (name.equalsIgnoreCase(registeredPatient)) {

    output.setText(
        "Error: Patient already registered.");
    return;
}

String selectedCategory =
    category.getSelectedItemAt();

String selectedDoctor =
    doctor.getSelectedItemAt();

String priority;

if (critical.getState() ||
    selectedCategory.equals("Critical")) {

    priority = "HIGH PRIORITY";
}

else if (selectedCategory.equals("Serious")) {

    priority = "MEDIUM PRIORITY";
}

else {

    priority = "NORMAL PRIORITY";
}

registeredPatient = name;

output.setText(
    "PATIENT REGISTERED\n\n" +
    "Patient: " + name + "\n" +
    "Category: " + selectedCategory + "\n" +
    "Doctor: " + selectedDoctor + "\n" +
    "Treatment Priority: " + priority);

if (priority.equals("HIGH PRIORITY")) {

    output.append(
        "\n\nALERT: Critical patient!");
}
}
}

```

Input

Patient Name: Ravi
 Emergency Category: Critical
 Doctor: Dr. Kumar
 Critical Patient: Yes

Output

PATIENT REGISTERED

Patient: Ravi
 Category: Critical

4. Smart Factory Monitoring and Alert Applet

Aim

To develop a Java Applet for monitoring temperature, pressure, machine status, and production level using **BorderLayout, GridLayout, CardLayout, MenuBar, and event listeners**.

Program

```
import java.applet.Applet;import java.awt.*;import java.awt.event.*;
public class SmartFactoryApplet extends Applet
    implements ActionListener {

    CardLayout cardLayout;
    Panel cardPanel;

    TextField temperature;
    TextField pressure;
    Choice machineStatus;
    Choice productionLevel;

    Button checkButton;
    TextArea output;

    MenuBar menuBar;
    Menu viewMenu;
    MenuItem monitorItem;
    MenuItem alertItem;

    public void init() {

        setLayout(new BorderLayout());

        // MenuBar
        menuBar = new MenuBar();

        viewMenu = new Menu("View");

        monitorItem = new MenuItem("Monitoring");
        alertItem = new MenuItem("Alerts");

        viewMenu.add(monitorItem);
        viewMenu.add(alertItem);

        menuBar.add(viewMenu);

        setMenuBar(menuBar);

        monitorItem.addActionListener(this);
        alertItem.addActionListener(this);

        // CardLayout
        cardLayout = new CardLayout();
        cardPanel = new Panel(cardLayout);
```

```

// Monitoring panel
Panel monitoringPanel =
    new Panel(new GridLayout(5, 2));

monitoringPanel.add(
    new Label("Temperature:"));

temperature = new TextField();
monitoringPanel.add(temperature);

monitoringPanel.add(
    new Label("Pressure:"));

pressure = new TextField();
monitoringPanel.add(pressure);

monitoringPanel.add(
    new Label("Machine Status:"));

machineStatus = new Choice();
machineStatus.add("Stopped");
machineStatus.add("Running");
machineStatus.add("Paused");

monitoringPanel.add(machineStatus);

monitoringPanel.add(
    new Label("Production Level:"));

productionLevel = new Choice();
productionLevel.add("Low");
productionLevel.add("Normal");
productionLevel.add("High");

monitoringPanel.add(productionLevel);

checkButton = new Button("Check Factory");

monitoringPanel.add(checkButton);

// Alert panel
Panel alertPanel =
    new Panel(new BorderLayout());

output = new TextArea();
output.setEditable(false);

alertPanel.add(
    new Label("Factory Alerts"),
    BorderLayout.NORTH);

alertPanel.add(
    output,
    BorderLayout.CENTER);

cardPanel.add(
    monitoringPanel,
    "Monitoring");

cardPanel.add(
    alertPanel,
    "Alerts");

```

```

        add(cardPanel, BorderLayout.CENTER);

        checkButton.addActionListener(this);
    }

    public void actionPerformed(ActionEvent e) {

        if (e.getSource() == monitorItem) {

            cardLayout.show(
                cardPanel,
                "Monitoring");
        }

        else if (e.getSource() == alertItem) {

            cardLayout.show(
                cardPanel,
                "Alerts");
        }

        else if (e.getSource() == checkButton) {

            checkFactory();
        }
    }

    void checkFactory() {

        try {

            double temp =
                Double.parseDouble(
                    temperature.getText());

            double press =
                Double.parseDouble(
                    pressure.getText());

            String machine =
                machineStatus.getSelectedItem();

            String production =
                productionLevel.getSelectedItem();

            output.setText("");

            if (temp >= 80 || press >= 100) {

                output.append(
                    "EMERGENCY MODE\n");
                output.append(
                    "Immediate attention required.\n");
            }

            else if (temp >= 60 || press >= 80) {

                output.append(
                    "WARNING MODE\n");
                output.append(
                    "Check factory conditions.\n");
            }
        }
    }

```



```

        else {

            output.append(
                "NORMAL MODE\n");
            output.append(
                "Factory operating normally.\n");
        }

        if (machine.equals("Running") &&
            production.equals("Low")) {

            output.append(
                "\nAlert: Low production level.");
        }

        if (machine.equals("Stopped") &&
            production.equals("High")) {

            output.append(
                "\nAlert: Conflicting machine status.");
        }
    }

    catch (NumberFormatException ex) {

        output.setText(
            "Invalid temperature or pressure.");
    }
}

```

Input

Temperature: 85
 Pressure: 90
 Machine Status: Running
 Production Level: High

Output

EMERGENCY MODE
 Immediate attention required.

Another Output

For:

Temperature: 40
 Pressure: 50
 Machine Status: Running
 Production Level: Normal

Output:

NORMAL MODE
 Factory operating normally.